C++ 多线程 Web 服务器开发手册

第一部分：先搞懂我们在干什么（概念篇）

在写代码之前，我们需要扫清这 4 个核心概念的障碍。

1. 什么是 Web 服务器?

想象你开了一家**餐厅**（服务器 Server）。

- **顾客**（浏览器/Client）：走进来点菜。
- **服务员**（你的程序）：记下顾客点的菜（接收请求），把菜端上来（发送响应）。
- 我们的目标：写一个程序，充当这个“服务员”。

2. 什么是 Socket (套接字)?

这是 Linux 网络编程最难懂的词。

- **比喻**：想象它是一个**电话座机**。
- 在 Linux 系统里，你想联网，就必须向系统申请一个“电话机”。系统会给你一个数字（比如 3），说：“拿去，这个 3 号就是你的电话机”。
- 以后你想发数据，就是对着 3 号说话；想收数据，就是听 3 号的声音。

3. 什么是 Bind (绑定) 和 Port (端口)?

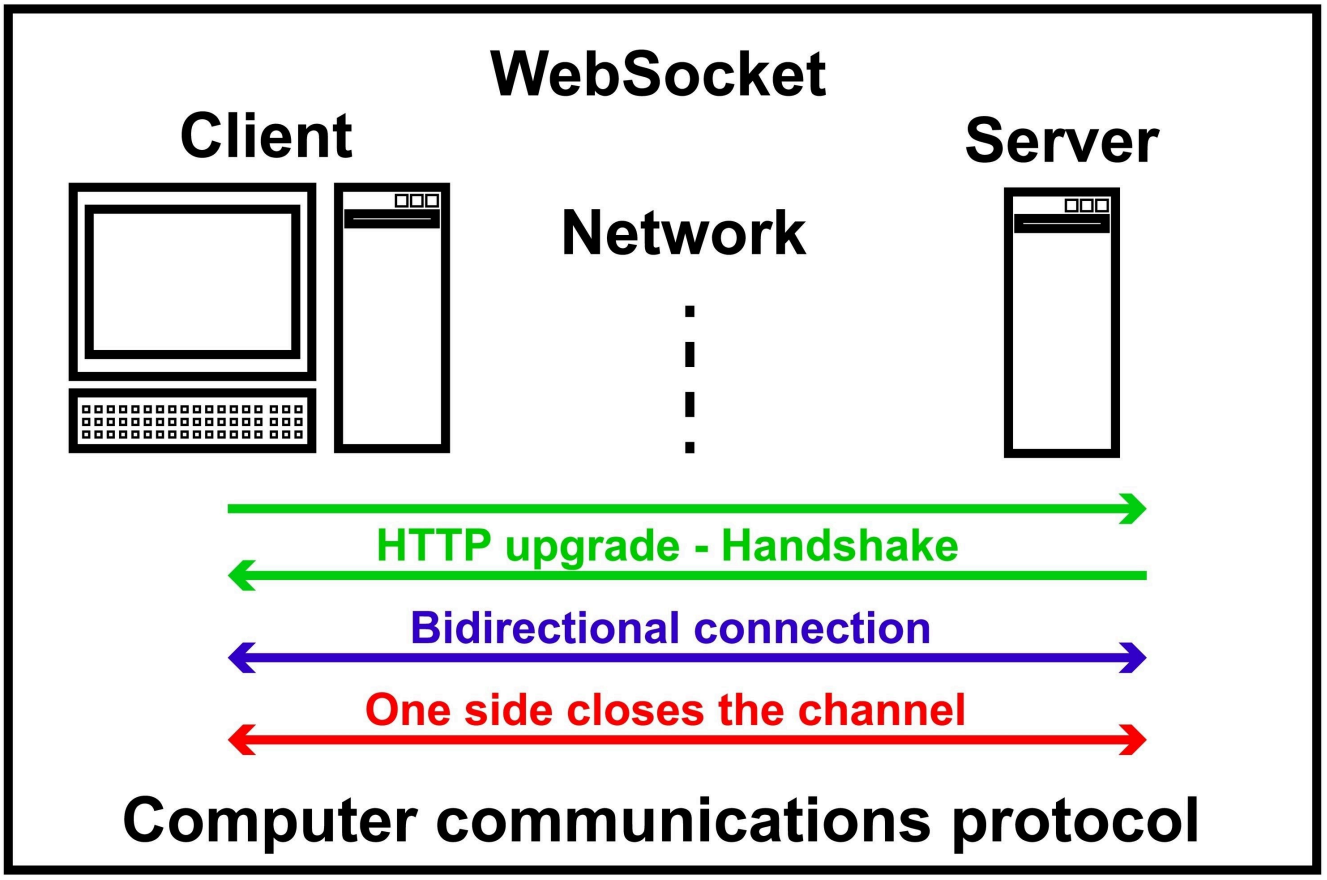
- **Bind (绑定)**：你买来了电话机（Socket），但别人怎么打给你？你得把电话线插在墙上的插孔里。
- **Port (端口)**：墙上有很多插孔，每个插孔都有编号（80, 8080, 443...）。
- **Bind 的过程**就是告诉系统：“把我的电话机（Socket）插在 **8080** 号插孔上。”这样，凡是打给 8080 的电话，都会转接到你的电话机上。

4. Listen (监听) & Accept (接受)：

- **Listen**：把电话设为“响铃模式”，随时准备接听。
- **Accept**：电话响了，你拿起听筒。此时，操作系统会**复制**一个新的 Socket 专门用来和这个人通话，原来的 Socket 继续等待下一个电话。

5. HTTP 协议：

- 这其实就是**纯文本**。浏览器给服务器发一段字：“我要看首页”；服务器回一段字：“好的，这是首页内容...”。我们写服务器，其实就是写一个程序来**解析和拼接字符串**。



6. 什么是 Header (.h) 和 Source (.cpp)?

C++ 代码通常分开放在两个文件里：

- `.h` **(头文件)**：像是**菜单**。它只列出这道菜叫什么（函数名），但不告诉你怎么做。
- `.cpp` **(源文件)**：像是**后厨的配方**。它写明了这道菜具体怎么炒（函数的具体代码逻辑）。
- **为什么要分开？** 为了整洁。别人想用你的代码，看菜单（.h）就够了，不用去研究配方。

🛠 第二部分：最详细的实战步骤（手把手）

现在我们开始动工。请完全按照我的步骤来，我会解释每一步的含义。

步骤 1：准备工作区

打开你的 Ubuntu 终端。

1. **新建文件夹**：

   Bash

   ```
   mkdir MyServer
   cd MyServer
   ```

2. **创建分类目录**（好的程序员必须有条理）：

   Bash

```
mkdir src       # 放配方 (.cpp)
mkdir include   # 放菜单 (.h)
mkdir build     # 放编译过程中的垃圾文件
mkdir www       # 放网页文件
```

步骤 2：配置 "工程管家" (CMake)

C++ 编译很麻烦，需要手动把一堆文件连在一起。我们用 CMake 这个工具来帮我们管理。

在 `MyServer` 根目录下，新建文件 `CMakeLists.txt`：

Bash

```
code CMakeLists.txt
```

复制进去，保存：

CMake

```
# 规定 CMake 最低版本，太老的版本不支持新特性
cmake_minimum_required(VERSION 3.10)

# 给这一堆代码起个项目名字
project(MiniServer)

# 告诉编译器我们要用 C++17 标准 (因为我们要用 std::thread 线程功能)
set(CMAKE_CXX_STANDARD 17)

# 告诉编译器：去 include 文件夹里找头文件(.h)
include_directories(include)

# 告诉编译器：去 src 文件夹里找所有的源代码(.cpp)，并把它们记在 SOURCES 变量里
file(GLOB SOURCES "src/*.cpp")

# 命令：把 SOURCES 里的代码编译成一个可执行程序，名字叫 server
add_executable(server ${SOURCES})

# 命令：Linux 下使用线程必须专门链接 pthread 库，否则会报错
target_link_libraries(server pthread)
```

步骤 3：写菜单 (server.h)

这个服务器的逻辑循环是：

1. **Socket**: 创建一个“电话机”。
2. **Bind**: 把电话机插在 `8080` 端口上。
3. **Listen**: 开始死等电话响。
4. **Accept**: 电话响了，接起来（建立连接）。
5. **Thread**: 叫一个分身（线程）去处理这个电话，主线程继续等下一个电话。

我们先定义服务器长什么样。 新建 `include/server.h`：

Bash

```
code include/server.h
```

复制进去（看中文注释！）：

C++

```
#ifndef SERVER_H  // 防止这个菜单被重复包含多次（固定写法）
#define SERVER_H

#include <netinet/in.h> // 这是 Linux 的网络工具箱
#include <string>

// 定义一个类，代表我们的服务器
class WebServer {
public:
    // 构造函数：启动服务器时，需要指定端口号（比如 8080）
    WebServer(int port);

    // 一个公开的方法：启动服务器
    void start();

private:
    // 下面这些是内部细节，外部不需要知道
    int server_fd; // 存放“电话机”的编号 (ID)
    int port;      // 存放端口号
    struct sockaddr_in address; // 一个结构体，专门用来填 IP 地址和端口

    // 处理客户请求的函数
    static void handle_client(int client_socket);
};

#endif
```

步骤 4：写配方 (server.cpp) —— 最难的一步

新建 `src/server.cpp`。这里是真正的逻辑。

Bash

```
code src/server.cpp
```

这一段很长，因为我加了巨详细的注释，请务必读一遍注释：

C++

```
#include "server.h"
#include <iostream>     // 输入输出流 (std::cout)
```

```
#include <unistd.h>     // Linux 系统函数 (read, write, close)
#include <cstring>      // 字符串处理
#include <thread>       // 线程库
#include <sys/socket.h> // Socket 核心库
#include <arpa/inet.h>  // 网络地址转换库

// --- 1. 服务器的初始化过程 ---
WebServer::WebServer(int port) : port(port) {
    // 第一步: 买电话 (Socket)
    // AF_INET = IPv4 协议
    // SOCK_STREAM = TCP 协议 (可靠传输)
    // 返回值是一个整数 ID, 如果 < 0 说明出错了
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("电话机买失败了 (Socket failed)");
        exit(EXIT_FAILURE); // 退出程序
    }

    // 第二步: 填写名片 (配置地址结构体)
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY; // 允许任何人打给我
    address.sin_port = htons(port);       // 把端口号转换成网络识别的格式

    // 第三步: 插电话线 (Bind)
    // 把 server_fd (电话机) 和 address (端口) 绑在一起
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("电话线插错了 (Bind failed)");
        exit(EXIT_FAILURE);
    }

    // 第四步: 设置响铃模式 (Listen)
    // 10 表示最多允许 10 个人同时排队, 后面的会占线
    if (listen(server_fd, 10) < 0) {
        perror("无法监听 (Listen failed)");
        exit(EXIT_FAILURE);
    }
}

// --- 2. 服务器的主循环 ---
void WebServer::start() {
    std::cout << "服务器启动了! 正在监听端口: " << port << std::endl;
    int addrlen = sizeof(address);

    while (true) {
        // 第五步: 接电话 (Accept)
        // 这是一个"卡住"的函数。如果没有人连上来, 程序会停在这里不动。
        // 一旦有人连上来, 它会返回一个新的 socket (new_socket)。
        // 注意: server_fd 是总机, new_socket 是分机, 专门用来跟这个人说话。
        int new_socket = accept(server_fd, (struct sockaddr *)&address,
(socklen_t*)&addrlen);

        if (new_socket < 0) {
            perror("接电话失败");
```

```
            continue; // 继续等下一个
        }

        // 第六步：找个临时工去处理 (多线程)
        // 创建一个新线程，去执行 handle_client 函数，处理这个 new_socket
        // .detach() 意思是：你尽管去干活，干完自己下班，不用向我汇报。
        std::thread(handle_client, new_socket).detach();
    }
}

// --- 3. 处理具体的对话 ---
void WebServer::handle_client(int client_socket) {
    char buffer[1024] = {0}; // 准备一个 1024 字节的空盘子，用来装数据

    // 从分机读取对方说了什么
    int valread = read(client_socket, buffer, 1024);
    if (valread <= 0) {
        close(client_socket);
        return;
    }

    std::cout << "收到消息:\n" << buffer << std::endl;

    // 准备回复的内容 (HTTP 协议格式)
    // 必须包含 HTTP/1.1 200 OK 这样的头，否则浏览器看不懂
    std::string response = "HTTP/1.1 200 OK\r\n"
                           "Content-Type: text/html\r\n\r\n"
                           "<h1>Hello! This is my first C++ Server</h1>";

    // 发送回复
    write(client_socket, response.c_str(), response.length());

    // 挂断电话
    close(client_socket);
}

void WebServer::send_response(int client_socket, const std::string& header, const
std::string& body) {
    write(client_socket, header.c_str(), header.size());
    write(client_socket, body.c_str(), body.size());
}
```

步骤 5：启动开关 (main.cpp)

新建 `src/main.cpp`：

C++

```
#include "server.h"

int main() {
    // 创建一个服务器实例，监听 8080 端口
    WebServer server(8080);

    // 按下启动按钮
    server.start();

    return 0;
}
```

4. 创建一个测试网页 `www/index.html`

HTML

```
<!DOCTYPE html>
<html>
<head>
    <title>My C++ Server</title>
</head>
<body>
    <h1 style="color: blue;">Hello from C++ Socket!</h1>
    <p>This page is served by a custom multi-threaded Linux server.</p>
</body>
</html>
```

🚀 第三部分：运行它 (The Build)

这是新手最容易懵的地方。我们刚才写的只是文本文件，要把它们变成电脑能跑的程序，需要**编译**。

1. **进入构建车间：** 我们之前建了一个 `build` 文件夹，专门用来放脏东西。

   Bash

   ```
   cd build
   ```

2. **召唤 CMake (生成说明书)：** 输入下面命令。注意后面有个空格和两点 `..`，意思是“源代码在上一级目录”。

   Bash

   ```
   cmake ..
   ```

   (这步完成后，你会发现 build 目录下多了 Makefile 和 compile_commands.json)

3. **开始制造 (Make)：**

   Bash

   ```
   make
   ```

成功标志：输出 [100%] Built target server(如果代码没问题，会生成一个叫 `server` 的可执行文件)

4. **准备环境并运行：** 我们的程序读取文件是写的相对路径 `www/index.html`，所以我们需要回到项目根目录去运行。

   Bash

   ```
   cd ..
   ./build/server
   ```

   你应该看到：服务器启动了！正在监听端口: 8080

🧪 第四部分：去浏览器看看

1. **浏览器验证：** 不要关闭终端，打开 Ubuntu 里的 Firefox，访问 `http://localhost:8080`。 你应该能看到蓝色的 "Hello from C++ Socket!"。
2. **压力测试（感受多线程的威力）：** C++ 的优势在于快。我们来压测一下。打开一个新的终端：
   - **安装压测工具：** `sudo apt install apache2-utils`
   - **发射：** `ab -n 1000 -c 10 http://localhost:8080/`
     - `-n 1000`: 发送 1000 个请求
     - `-c 10`: 10 个并发线程同时发
   - **看结果：** 观察 **Requests per second** (RPS)。在本地回环下，这个简单的 C++ 服务器通常能跑到几千甚至上万 RPS。

🎓 极客笔记：你到底学到了什么？

1. **文件描述符 (File Descriptor)：**
   - 在 Linux 看来，`client_socket` 只是一个整数。读取网络数据和读取文件（`read/write`）用的是同一套 API。这就是 Unix 哲学：**“一切皆文件”**。
2. **HTTP 协议本质：**
   - 没有什么魔法，HTTP 仅仅是按照特定格式（Headers + 空行 + Body）拼接起来的**纯文本字符串**。你自己拼出来的字符串，浏览器就能看懂。
3. **CMake 工程化：**
   - 你学会了用 `compile_commands.json` 配合 `clangd`。现在打开 VS Code，你会发现代码跳转和补全依然完美工作，这就是构建系统的作用。
4. **并发模型：**
   - `std::thread(...).detach()` 是最基础的并发。每个请求进来，Linux 内核都会为你调度一个新的线程。虽然这比 Python 的单线程快，但如果在高并发下（比如 10000 个人同时连），创建 10000 个线程会把内存撑爆。
   - *进阶思考*：这就是为什么 Nginx 使用 **IO 多路复用 (epoll)** 而不是简单的多线程。

深度原理解析 (面试考点)

为了让你知其所以然，这里有几个核心概念的通俗解释：

1. **为什么需要 bind?**
    - Socket 刚创建时只是一个“插座”。`bind` 把它和 `8080` 端口连起来，这样外面发给 8080 的数据才会流进这个 Socket。
2. **accept 是干嘛的?**
    - 主 Socket (`server_fd`) 像一个**前台接待员**，它只负责接电话，不负责具体聊天。
    - 当有人打进来，`accept` 会制造一个**分身** (`new_socket`) 给这个客户。
    - 主 Socket 立刻挂断，继续等下一个人的电话；分身负责和客户聊具体的网页内容。
3. **多线程 detach 的意义?**
    - 如果不使用线程，你的服务器处理一个请求时（比如传一个大图片），其他所有人都得排队，这就是**阻塞**。
    - 开了线程，就像**雇了临时工**。
    - `detach` 意味着告诉系统：“这个临时工干完活自己走人，不用向我（主线程）汇报工作结果”。这是防止内存泄漏的最简方法。

下一步挑战： 如果这个跑通了，试着修改 `handle_client`，解析 buffer 里的字符串，让它可以根据请求不同的 URL (`/image.jpg`, `/about.html`) 读取不同的文件？